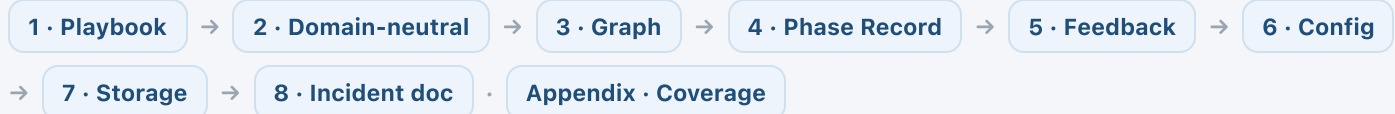


Investigation Engine — Data Model

Domain-neutral by design — one engine, many playbooks. The shapes, the run loop, the registry, feedback and storage are universal; each **playbook** supplies its own domain (types · phases · outputs). Shown end-to-end with the **Incident-Triage** playbook. Every schema is *defined* → *explained* → *shown as JSON* → *tabled*.

v1.0 · consolidated from the HLD + LLD + the 14-incident validation (G1–G25) · coverage-checked (Appendix) · written so a human can approve each schema and understand every field — no ambiguity.



1 · The Playbook

The self-sufficient domain reference — process + types, tool-agnostic

We start here because this is the part we agree on — and because **this file is the only place anything domain-specific lives**. Front-matter the engine **executes** — `phases` (intent + effect, **no tools**), `graph_schema` (the domain **type system**), `output_schemas` — and a body it **follows**. Swap this file → new domain, same engine (§2 shows exactly how). Stored as a config row in §6.1; every type and field defined in §3–§4.

```
---
id: incident-triage
version: 1.0.0
domain: app-incident

defaults:
  on_failure: run-remaining          # finish independent steps, then report
  retry: { max: 3, backoff: exponential } # TRANSIENT errors only; permanent → error_handler
  unknown_access: ask
  error_handler: { action: escalate, to: on-call, via: pagerduty }

phases:                                # tool-agnostic: intent (needs) + effect, never a
- { id: assess,          effect: read-only, output: AssessResult,
  goal: "Know what's affected, how bad, what changed – surface impact + early suggestions.",
  needs: [incident-source, topology, change-history, telemetry, similar-incidents] }
- { id: root-cause,      effect: read-only, output: RootCauseResult, min_confidence: 0.7,
  goal: "Find the most probable cause – ranked, with evidence; pivot if the health-walk dead
  needs: [metrics, logs, traces, topology, change-history, layer-deep-dive] }
- { id: remediation,     effect: write, gate_writes: true, output: RemediationResult,
  goal: "Apply the safest fix – reverse, mitigate, or escalate; stop the bleeding.",
  needs: [remediation-action, escalation] }
- { id: verify-close,    effect: read-only, output: VerifyResult,
  goal: "Confirm recovery from the user's side; revert temporaries; a human closes.",
  needs: [telemetry, synthetic-replay] }
```

```
# the engine maps `needs` (intents) → concrete capabilities from the LIVE registry (§6.2), bound

graph_schema:                                     # THE DOMAIN TYPE SYSTEM – incident-triage's voca
  node_types:
    system: { business: [business_service], app: [app, service, component], database: [database]
              network: [network_device], storage: [storage], compute: [compute],
              location: [zone, region], external: [external_service], any: [generic] }
    incident: [incident, cluster]                  # exactly ONE `incident` node per graph (the subj
    change:   [change]
    alert:    [alert]
  edges: [part_of, depends_on, runs_on, hosted_on, routes_to, located_in, changed, affects,
          suspected_cause, related_to, member_of, observed_on, correlated_into]
  facts: [health, error_rate, latency, sessions, deadline, impact_state]
  labels: [suspect, policy_block, central, pci, tier-0]
  conventions: [policy_block, realized_by, alert_noise]

output_schemas:                                  # this playbook's phase output contracts (every f
  AssessResult:   { required: [incident_type, symptom, affected, impact_assessment, changed,
  RootCauseResult: { required: [candidates, selected, status] }
  RemediationResult: { required: [actions] }
  VerifyResult:   { required: [recovered, resolution] }

---

## assess
Build the picture – read-only. From the incident-source seed the incident + alert nodes; from to
load the affected service(s) + neighbourhood; overlay light health on the symptom; pull recent c
link related / similar priors and USE them. Cluster if incidents converge on one upstream node o
fingerprint in a window. Emit the impact_assessment + early suggestions (from similar past incid
`changed:[]` is a first-class "checked-and-clean" assertion; record a `time_factor` for time-tri

## root-cause
Ranked candidates – never one asserted – each with a `suspected_cause.path` (victim → ... → cause)
confidence whose basis is stated. Walk the graph; rule out alternatives with evidence. Fal
if no node is unhealthy (data / async-lag / expiry read green on health), pivot – data-quality a
`changed:[]` + `time_factor`, write-edges, and `impact_state`. Cause may be a node, change, zone

## remediation
The safest fix – every write gated. `kind`: reverse · mitigate (technique; temporary ones carry
`revert_when`) · escalate (names the team). Each action carries `expected_effect`, `blast_radius
`rollback`. Idempotency key (exactly-once); permanent failure → error_handler. `followups[]` for

## verify-close
Confirm recovery from the user's side (before/after + synthetic replay). Confirm every `temporar
is reverted-or-scheduled before close. Record the resolution. If it didn't hold, backtrack t
Human closes.
```

2 · Domain-neutral by design universal engine

One engine, many playbooks — no domain lock-in

The model has **three levels**. Only the middle one is domain-specific — so the same engine runs incident-triage today and provisioning or capacity tomorrow, by loading a different playbook.

Level	What it is	Domain-specific?
-------	------------	------------------

Engine	the shapes (Node·Edge·Fact·PhaseRecord·Step), <code>govern()</code> , the run loop, gates, retries, clustering, the registry, feedback, the two storage planes	No — universal. Never changes between domains.
Playbook	the type system (<code>graph_schema</code>), the phases + their <code>needs</code> / <code>effect</code> , and the output contracts (<code>output_schemas</code>)	Yes — this is the <i>only</i> domain layer. One file per domain.
Runtime	the concrete nodes/edges/records of one investigation	Instances — conform to whatever playbook is loaded.

What is universal vs what each playbook supplies

Universal (engine — any domain)	Per-playbook (the domain)
Node / Edge / Fact shapes (id, kind, type, labels, props, facts; from/to/props)	the values : <code>node_types</code> , edge types, fact keys, labels, conventions
PhaseRecord / Step shapes; the run loop, gates, retries, backtrack, clustering	the phases (id · goal · <code>needs</code> · <code>effect</code>) and how many there are
the <code>PhaseRecord.output</code> slot	the typed outputs (AssessResult... for incidents; ValidationResult... for provisioning)
Provider · DeclaredCapability · CapabilityPolicy · Feedback · the two storage planes · the read-model doc shape	which capabilities a playbook's <code>needs</code> resolve to
the subject reference — how every engine record names the thing under investigation: <code>{domain, id, kind}</code>	the subject's kind + its id format (incident → <code>INC-...</code> ; provisioning → <code>REQ-...</code>)

How the subject is identified — domain-neutral (never a bare `incident_id`)

Why a subject reference, not `incident_id` : the PhaseRecord (§4), Feedback (§5), the run, and the read-model must point at the thing under investigation *without* assuming it is an incident. So they carry a **SubjectRef** — the pair `{domain, id}` (plus the subject's `kind`). The unique key is **(domain, id)**: incident-triage fills it `{app-incident, INC-4821}`, provisioning `{provisioning, REQ-123}`. This is what makes the neutrality *real*, not just claimed.

```
{
  "domain": "app-incident", // SubjectRef — used by PhaseRecord (§4.1), Feedback (§5), th
  "id": "INC-4821", // = Playbook.domain — which world this is
  "kind": "incident" // the subject's identity IN that domain (provisioning: "REQ-
// the subject node's kind (provisioning: "provision_request"
} // unique key = (domain, id) — the engine never assumes the w
```

Proof — the same shapes carry a different domain. A second playbook changes only the green/right column above. Everything else is identical.

```
id: provisioning-request          # a DIFFERENT investigation playbook – SAME engine, SAME
domain: provisioning
phases:
  - { id: validate,      effect: read-only, output: ValidationResult, needs: [request-source]
  - { id: allocate,      effect: write, gate_writes: true, output: AllocationResult, needs:
  - { id: verify-close, effect: read-only, output: VerifyResult,      needs: [resource-health]
graph_schema:
  node_types:                  # totally different vocabulary – but still Nodes
    request: [provision_request]
    resource: [vm, database, bucket, network]
    actor: [requester, approver]
    quota: [quota]
  edges: [requires, consumes, approved_by, blocked_by, allocated_to] # still Edges
  facts: [quota_available, cost_estimate, policy_check]              # still Facts
  output_schemas: { ValidationResult: {...}, AllocationResult: {...}, VerifyResult: {...} }
```

The engine never learns the word "incident" or "vm" — it only sees *nodes with a kind from the loaded playbook's vocabulary*. That is what "no domain lock-in" means: a library of playbooks, one engine.

3 · The Investigation Graph Mongo · data plane

The per-investigation world: Node, Fact, Edge

One graph per investigation — the shared world all phases read and write. The **shapes** here are universal (§2); the **type/edge/fact values** shown are incident-triage's, from its `graph_schema`. **NEW** = a field the phase-output sufficiency check added.

The incident & related-incidents rule — one place, stated once. A graph has **exactly one subject node** — in incident-triage, the `incident` node (provisioning's would be a `provision_request`): the subject of *this* investigation. But **other incidents are an active part of the work** — during Assess we pull in *related* and *similar* priors and **use** them: to **correlate** (do they share a fingerprint or converge on one node?), to **carry knowledge** (a similar incident's `resolution` becomes a candidate suggestion), and to **link** them (`related_to`, `kind = child` || `related` || `caused_by` || `chained` || `similar`). They are **referenced, not duplicated** — we hold an `id` + `kind` + the few bits we actually use (its `resolution`, `fingerprint`, `cause`); the full incident stays in its own document. When the engine spots incidents that share a cause, it **surfaces them to the operator**, who may **link** them into a **cluster** (each via `member_of`) — a human-confirmed grouping, **not a separate run**. Each incident keeps its **own session, phases, and closure**; the cluster just lets the shared cause/fix propagate. So `incident: [incident, cluster]` = *the one subject, plus the optional cluster it may be linked into*. One playbook (incident); no "cluster playbook."

3.1 · Node — a thing in the investigation

Definition. A typed vertex in the graph. The shape is universal; `kind/type/layer` draw their *values* from the loaded playbook's `node_types` (here, incident-triage's).

Why this shape: one node shape for every kind so the engine walks a single uniform graph in any domain. `kind → type → layer` is the only branching the engine needs (it tells the deep-dive which tools apply). `labels` and `props` stay open so a new source or attribute needs **no schema change**.

```
{
  "id": "app:payments-api",           // kind:name – stable; everything refers to a node by id
  "kind": "system",                  // value from the playbook's node_types (incident-triage:
  "type": "app",                     // a type within the kind (or "generic")
  "layer": "app",                     // system only: business|app|database|network|storage|com
  "name": "Payments API",
  "labels": ["pci","tier-0","suspect"], // open tags; some (suspect/policy_block/central) drive
  "props": { "owner_team":"payments-sre", "on_call":"j.rivera" },
  "facts": [ /* Fact[] – observations made during the run; see 3.3 */ ],
  "summary": "degraded since 14:02; DB-backed latency",
  "sources": ["servicenow-cmdb","appd"] // which tools contributed this node
}
```

Field	Type / values	What it means
id	string · unique	Stable identifier, convention <code>kind:name</code> . Edges and the cause-path refer to nodes by id.
kind	from <code>playbook node_types</code>	Top category. (<i>incident-triage</i> : system = a real thing in the estate; incident = the one subject of this graph; change = a deploy/config/flag; alert = a fired signal.)
type	a type within the kind, or <code>generic</code>	The specific type (system→ <code>app</code> , system→ <code>database</code>). <code>generic</code> = the escape when nothing fits. <i>Full vocabulary in 3.2.</i>
layer	<code>business·app·database·network·storage·compute·location·external</code>	system nodes only . Which layer of the stack — tells the engine which deep-dive applies.
name	string	Human-readable label.
labels[]	string[] (open)	Runtime tags. Conventional ones drive triage — <code>suspect</code> , <code>policy_block</code> , <code>central</code> — plus free tags.
props{}	open map	Source-specific attributes + ownership: <code>owner_team</code> , <code>on_call</code> , <code>support_group</code> .
facts[]	<code>Fact[]</code> (embedded)	Observations made about this node during the run (3.3).
summary · sources[]	string · string[]	Rolling summary of the node's state · which tools contributed it.

3.2 · Node types — incident-triage's value-space for `kind` / `type` / `layer`

Why a fixed vocabulary (per playbook): the engine only ever branches on these — so a playbook keeps them closed and few. `generic` is the one open door, so an unknown thing is still a first-class node (never dropped) and can be promoted later. (*A provisioning playbook would list request/resource/actor/quota instead — §2.*)

kind	layer	type	Represents
system	business	business_service	A business service spanning apps ("Checkout Journey") — where impact is felt and reported.
system	app	app · service · component	An application; a (micro)service within it; a finer component (gateway, library).
system	external	external_service	A third-party / shared dependency — a vendor API, a partner endpoint.
system	database	database	A database instance or cluster.
system	network	network_device	A load balancer, switch, router, firewall, or DNS (which one in a label).
system	storage	storage	A volume, datastore, or array.
system	compute	compute	A pod, host, cluster, or VM (which one in a label).
system	location	zone · region	An AZ / region — lets the engine correlate <i>co-location</i> faults, not just dependency faults.
system	any	generic	The escape hatch — anything with no known type (a cache, an ETL job). Promote if it recurs.
incident	—	incident · cluster	incident = the one subject of this graph (exactly one). cluster = an <i>operator-confirmed</i> grouping of related incidents that share a cause (via <code>member_of</code>) — a link, not a separate run. <i>See the rule above.</i>
change	—	change	A deploy, config change, feature-flag flip, or change ticket (which one in a label).
alert	—	alert	A fired alert — its own node because it has identity + a lifecycle and may attach to many nodes.

3.3 · Fact — an observation on a node

Definition. A single timestamped, sourced observation embedded on a node (its health, a metric, an alert reading).

Why a property on the node, not its own node: an observation has no identity worth a node — embedding it keeps the whole picture of a node in one place to read. `evidence_ref` makes it **auditable** (a deep link, not hearsay). There is **no ttl** — freshness is judged at read time. `impact_state` is separate from `health` so a *healthy-but-impacted* node still counts toward blast radius.

```
{
  "key":      "health",           // fact key from the playbook (incident: health|err
  "value":    "degraded",
  "source":   "appd",            // the tool that observed it
  "evidence_ref": "appd://payments-api/bt?t=14:02", // deep link to backing data – auditable
  "observed_at": "2026-06-18T14:02Z",           // NO ttl – freshness judged at read time vs what t
  "confidence": 0.9,                // 0–1; discounted for noisy/suppressed sources
```

```
"impact_state": "degraded" // erroring|degraded|stalled|lagging|wrong-data|ok
}
```

Field	What it means
key · value	What's observed + the value — the keys are the playbook's (<code>health</code> , <code>error_rate</code> , <code>latency</code> ...).
source	The tool that observed it.
evidence_ref	A deep link to the backing data (log query · trace id · dashboard) — auditable, not hearsay. <i>(This is why no separate Evidence entity is needed — see Appendix.)</i>
observed_at	No ttl. Freshness is judged here, at read time, against what the reader needs.
confidence	0–1; discounted for noisy / suppressed sources.
impact_state NEW	Distinct from <code>health</code> . A <code>health:ok</code> node can still be impacted — <code>lagging</code> , <code>wrong-data</code> — and counts toward blast radius.

3.4 · Edge — a typed relationship

Definition. A directed, typed link between two nodes — dependency, containment, causation, correlation.

Why typed edges + open props : the `type` carries the meaning the engine reasons over (walk `depends_on` for blast radius, `suspected_cause` for the cause path); `props` carry the per-type extras without exploding into dozens of subtypes. Direction is meaningful — the two walks go opposite ways.

```
{
  "type":    "suspected_cause",      // edge type from the playbook (see 3.5)
  "from":    "INC-4821",
  "to":      "stor:pay-vol",         // direction matters: blast-radius and cause-finding wal
  "props": {
    "confidence": { "value":0.9, "basis":"evidence-complete + matches INC-4820" },
    "rank":       1,
    "path":       ["app:payments-api","db:payments-ora","stor:pay-vol"] // the causal chain vi
  },
  "sources": ["otel","netapp"]
}
```

Field	What it means
type	The relationship (full list + meanings in 3.5).
from · to	The two node ids. Direction matters — blast-radius walks edges one way, cause-finding the other.
props{}	<code>confidence</code> · <code>last_seen</code> · <code>via</code> (cmdb trace) · <code>kind</code> (related_to) · <code>rank</code> · <code>realized_by</code> · <code>path</code> (causal chain on <code>suspected_cause</code>).
sources[]	Tools that asserted the edge.

3.5 · Edge types — incident-triage's relationships

edge	connects (from → to)	Meaning
part_of	component→service→app ; pod→cluster	containment — the child is part of the parent
depends_on	app/service → service database external network_device	a logical dependency — the source needs the target to function
runs_on	service → pod → host	deployment — where the thing executes
hosted_on	database host → storage	what storage backs it
routes_to	service → network_device → ... → service	the network path between two points
located_in	host → zone → region	physical locality — for AZ / rack co-location correlation
changed	change → (system node)	a change touched this node (carries the time)
affects	incident → (system node)	the incident impacts this node
suspected_cause	incident → (node change zone)	the ranked probable cause; props carry confidence{value,basis}, rank, path
related_to	incident → (other incident cluster)	links to other incidents — referenced + actively used (not loaded wholesale); kind = child \\ related \\ caused_by \\ chained \\ similar
member_of	incident → cluster	this incident is linked into a shared-cause cluster (operator-confirmed)
observed_on	alert → (system node)	an alert fired on this node (one alert may attach to many)
correlated_into	alert → (incident cluster)	the correlation engine grouped this alert into the incident/cluster

3.6 · Conventions & clustering — reasoning rules on top of the shapes

Why conventions, not new fields: these are *reasoning rules* the engine applies to the shapes above — they need only a label or a prop, so the model stays small.

Rule	What it means
policy_block	A node <i>healthy on its own signals but the cause</i> (a firewall enforcing a bad rule, an expired cert, a node serving bad data). Rule: a changed -edge / fact / time_factor on a <i>healthy</i> node on the victim's path → treat it as suspect despite health.
realized_by	depends_on.props.realized_by:[net...] names the physical path that implements a logical dependency — so blast-radius (logical) and fault-localization (physical) reconcile.
alert_noise	Noisy / suppressed alerts count toward <i>clustering</i> (volume + simultaneity) but are <i>discounted</i> in root-cause confidence — a storm can't outvote a high-confidence fact.

cluster (operator-linked)

When incidents **share an alert fingerprint** AND their walks **converge on one node** in a window, the engine **surfaces the match to the operator** → who **links** them into a `cluster ( member_of )`. Never auto-merged; nothing runs "at cluster scope" — each incident keeps its own session.

4 · The Phase Record Mongo · data plane

The per-phase journey: PhaseRecord, Step, and the four outputs

One record per phase entered — the durable account of what the agent did, written into the subject document on close (the live run state itself stays in LangGraph, §7). The PhaseRecord and Step shapes are universal; the **four outputs are incident-triage's** (another playbook defines its own).

Output contracts are enforced: each phase output is validated as a Pydantic model (AssessResult, RootCauseResult, RemediationResult, VerifyResult) — the `output_schemas` in the playbook are descriptive; these models are the enforced contract.

4.1 · PhaseRecord — the per-phase journey

Definition. The plan, the ordered steps, and the typed output for one phase of one investigation.

Why separate from the graph: the graph is the shared *world*; the PhaseRecord is the *journey* (what the agent did *this* phase). Same world, many journeys. `steps[]` is the audit trail; `output` is the typed contract the next phase reads. They connect through each step's `touched[]` (→ graph) and `evidence[]` (→ output).

```
{
  "id": "INC-4821:root-cause:1", // subject.id:phase:attempt
  "subject": { "domain": "app-incident", "id": "INC-4821", "kind": "incident" }, // SubjectRef
  "phase": "root-cause",
  "goal": "Find the most probable cause – ranked, with evidence.", // from the playbook
  "state": "done", // active|waiting_input|waiting_approval|blocked|done
  "plan": "Walk below the app; correlate vs rev47; rank with a path.",
  "steps": [ /* Step[] – the ordered, timestamped journey; see 4.2 */ ],
  "output": { /* the typed phase output – points back into the graph; see 4.4 */ },
  "summary": "Cause = pay-vol disk failure (storage), conf 0.9.",
  "opened_at": "14:04", "closed_at": "14:07" // closed_at = null while open
}
```

Field	What it means
id	<code>subject.id:phase:attempt</code> . The attempt number lets a phase be re-run after a backtrack.
subject	the SubjectRef <code>{domain, id, kind}</code> (§2) — the one incident this record belongs to; never a bare <code>incident_id</code> . (Each incident has its own records — no "cluster scope".)
phase · goal	Which phase · its goal (carried from the playbook).
state	active · waiting_input · waiting_approval · blocked · done · failed.

plan · steps[] · output	What it set out to do · the ordered journey (4.2) · the typed result (4.4).
opened_at · closed_at	ISO8601 timestamps. opened_at set when the record is created; closed_at set on every terminal state (done, failed, blocked) — null only while active.
summary	Rolling summary of the phase's work.

4.2 · Step — one entry in the journey

Definition. One ordered event in a phase — a tool call, a reasoning note, a suggestion, the gate decision, or operator input.

Why one shape with a kind discriminator: the journey is a *single ordered log* — so the audit reads top-to-bottom with no joins. This deliberately keeps as **one** record what the HLD split into three tables (Action + HumanInteraction + RunEvent); the gate decision is just a step of `kind:"decision"`. `touched[]` and `evidence[]` tie a step to the **graph** and to the **output**.

```
{
  "seq":      2, "at": "14:04:30",
  "kind":     "tool_call",           // tool_call | reasoning | suggestion | decision | us
  "capability": "traces",           // resolved from a `need` via the registry (§6.2), NE
  "input":    { "trace":"checkout" },
  "result":   { "db_span_ms":3140, "pct_of_total":"75%" },
  "touched":  ["svc:checkout","db:payments-ora"], // → graph node/fact ids read or written
  "evidence": ["otel://trace/9af3"], // → deep links that flow up into the output
  "note":     "75% of latency is the DB call – not app code"
}
// other kinds: suggestion → { proposal, rationale }  decision → { decision, actor }  user_inp
```

Field	What it means (by kind)
seq · at · kind	(all) order · ISO8601 timestamp (every step) · the kind ∈ {tool_call, reasoning, suggestion, decision, user_input}.
capability · input · result	<i>tool_call</i> — the capability invoked (resolved from a <code>need</code> , never a hardcoded tool) + args + what came back. tool_call steps MUST name the capability id.
touched[]	<i>tool_call</i> — graph node/fact ids read or written → links the step to the graph.
evidence[]	<i>tool_call</i> — deep links to backing data → flows into the output.
proposal · rationale	<i>suggestion</i> — what's proposed + why.
decision · actor	<i>decision</i> (the gate) — approve \ refine \ deny + who.
message · note	<i>user_input</i> — what the operator said · (any kind) a short reasoning note.

4.3 · Worked example — a Root-cause PhaseRecord, fully connected

Why a filled one: the schemas above show the *shape*; this shows them *working together* — the chain steps → graph → output, end to end.

```

{
  "id": "INC-4821:root-cause:1", "subject": {"domain":"app-incident","id":"INC-4821","kind":"inc
  "phase": "root-cause", "state": "done",
  "goal": "Find the most probable cause – ranked, with evidence.",
  "plan": "Walk below the app from Assess's hint; overlay health; correlate vs rev47; rule out;
  "steps": [
    { "seq":1, "at":"14:04:10", "kind":"reasoning",
      "note":"Assess flagged the DB backend slow – walk below the app, not the app code" },
    { "seq":2, "at":"14:04:30", "kind":"tool_call", "capability":"traces",
      "input":{"trace":"checkout"}, "result":{"db_span_ms":3140,"pct_of_total":"75%"},
      "touched":["svc:checkout","db:payments-ora"], "evidence":["otel://trace/9af3"], // touch
      "note":"75% of latency is the DB call – not app code, not gemini" },
    { "seq":3, "at":"14:05:00", "kind":"tool_call", "capability":"metrics(db)",
      "input":{"target":"db:payments-ora"}, "result":{"io_wait_ms":28,"plan":"unchanged","locks"
      "touched":["db:payments-ora"], "evidence":["oem://payments-ora/awr?t=14:05"],
      "note":"DB slow from I/O latency, not SQL/locks → look below the DB" },
    { "seq":4, "at":"14:05:30", "kind":"tool_call", "capability":"metrics(storage)",
      "input":{"target":"stor:pay-vol"}, "result":{"latency_ms":22.4,"disk":"reconstructing sinc
      "touched":["stor:pay-vol"], "evidence":["netapp://aggr01/disk/1.4.7"],
      "note":"R00T – failed disk → RAID rebuild → vol latency → DB I/O → app; 13:58 aligns with
    { "seq":5, "at":"14:05:50", "kind":"tool_call", "capability":"metrics(network),metrics(compu
      "result":{"network":"clean","compute":"clean"}, "touched":["net:alb-pay","pod:checkout-7f9
      "note":"rule out network + compute" },
    { "seq":6, "at":"14:06:10", "kind":"reasoning",
      "note":"rule out rev47 (trace shows DB cost, not app code) and gemini (429s = downstream r
    { "seq":7, "at":"14:06:30", "kind":"suggestion",
      "proposal":"Cause: pay-vol disk failure → RAID rebuild → DB I/O. Confidence 0.9.",
      "rationale":"disk-reconstructing + I/O-wait + the trace's DB span; matches INC-4820" },
    { "seq":8, "at":"14:07:00", "kind":"decision", "decision":"approve", "actor":"j.rivera (sre-
  ],
  "output": {
    // RootCauseResult – built from the steps above
    "candidates": [ { "cause":"pay-vol disk failure → RAID rebuild → DB I/O", "node":"stor:pay-v
      "confidence":{"value":0.9,"basis":"evidence-complete + matches INC-4820"}, "rank":1,
      "path":["app:payments-api","db:payments-ora","stor:pay-vol"],
      "evidence":["otel://trace/9af3","oem://payments-ora/awr?t=14:05","netapp://aggr01/disk/1
    "selected": 0,
    "ruled_out": [ {"hyp":"rev47 deploy","evidence":"trace shows DB cost, not app code"},
      {"hyp":"gemini-api","evidence":"429s are downstream retries"} ],
    "status": "confident"
  },
  "summary": "Cause = pay-vol disk failure (storage), conf 0.9, path app-db-storage. rev47/gemin
  "opened_at":"14:04", "closed_at":"14:07"
}
// the chain: step.touched[] → graph nodes · step.evidence[] → output.candidates[].evidence[] ·
// graph side-effect on commit: stor:pay-vol gains label `suspect`; edge INC-4821 -suspected_ca

```

4.4 · The four phase outputs — *incident-triage's* contracts

Why a typed output per phase: it's the hand-off — the next phase, the UI, the audit, and the learning loop all read *this*, not the raw steps. These four are this playbook's `output_schemas`; a provisioning playbook would define `ValidationResult` / `AllocationResult` / `VerifyResult` instead. **NEW** marks a field the sufficiency check added.

AssessResult — the framed situation

```
{
  "incident_type": "performance",          // perf|availability|data|network|capacity|change|bus
  "symptom":      "checkout p99 0.4s-4.2s, errors 18%",  // the anchor for everything downstre
  "affected":     ["app:payments-api", "biz:checkout-journey"],
  "impact_assessment": { "scope": "customer-facing", "blast_radius": ["app:payments-api"],
                        "bounded_by": null, "severity": "P1", "urgency": "high",
                        "business_impact": "checkout payments degraded; revenue path" },
  "changed":     ["chg:deploy-rev47"],        // [] = checked & clean (NOT "unknown")
  "time_factor":  null,                      // or { kind:cron|scheduled_expiry|ttl_lapse|lice
  "suspected_locus": { "node": "db:payments-ora", "why": "DB span dominates" }, // head start for
  "related":     ["INC-4820"], "cluster": null, // related incidents actively used (suggest
  "suggestions": [ { "possible_fix": "failover DB to standby", "basis": "similar_incident", "con
  "owner":       "payments-sre"
}
```

Field	What it means
incident_type NEW · symptom NEW	The classification (drives traversal + UI badge) · the one-line "what's wrong" (the anchor downstream).
affected[]	The nodes (and business service[s]) in scope.
impact_assessment	<code>scope</code> · <code>blast_radius</code> (+ optional <code>bounded_by</code> = a change's scope) · <code>severity</code> · <code>urgency</code> · <code>business_impact</code> .
changed[] · time_factor?	Recent changes (<code>[]</code> = checked & clean) · a time-triggered contributor that isn't a change record.
suspected_locus? NEW	A tentative first suspect {node, why} — a head-start for Root cause.
related[] · cluster? NEW	Linked incidents (actively used for suggestions/correlation); the cluster id if part of one.
suggestions[] · owner	Early fixes {possible_fix, basis: similar_incident playbook, confidence} from priors · owner_team/on_call.

RootCauseResult — the ranked cause

```
{
  "candidates": [ {
    "cause": "pay-vol disk failure → RAID rebuild → DB I/O", "node": "stor:pay-vol",
    "confidence": { "value": 0.9, "basis": "evidence-complete + matches INC-4820" }, // basis = W
    "rank": 1, "path": ["app:payments-api", "db:payments-ora", "stor:pay-vol"],
    "evidence": ["otel://trace/9af3", "netapp://aggr01/disk/1.4.7"], "recommended_fix": "failover D
    "selected": 0, // the candidate being acted on (after the operator
    "ruled_out": [ { "hyp": "rev47 deploy", "evidence": "trace shows DB cost, not app code" } ],
    "status": "confident" // confident | needs-input | escalated
  }
]
```

Field	What it means
candidates[]	Ranked causes. Each: <code>cause</code> · <code>node</code> (suspect) · <code>confidence{value, basis}</code> NEW · <code>rank</code> · <code>path</code> · <code>evidence[]</code> · <code>recommended_fix?</code> NEW .

selected NEW	The candidate being acted on (after the operator confirms) — Remediation reads this.
ruled_out[] · status	Rejected hypotheses + the killing evidence · confident \\ needs-input \\ escalated.

RemediationResult — the gated fix

```
{
  "actions": [ {
    "action_id":"a1", "kind":"mitigate",          // reverse | mitigate | escalate
    "technique":"failover",                      // mitigate: failover|reroute|throttle|scale|is
    "target":"db:payments-ora", "team":null,      // team set only for escalate actions
    "expected_effect":"DB served by standby; I/O normal",  "blast_radius":"payments read path",
    "rollback":"fail back once disk replaced",
    "temporary":true, "revert_when":"incident_close",  // incident_close | action:<id> | proble
    "idempotency_key":"INC-4821-a1", "gated":true,
    "approval":{" decision":"approve", "actor":"j.rivera", "at":"14:21" },
    "result":"applied; I/O 28ms→4ms", "status":"done" } ],
  "followups": [ { "detail":"replace disk 1.4.7", "basis":"escalate", "owner":"storage" } ],
  "status": "applied"                                // applied | partial | failed
}
```

Field	What it means
actions[]	Each action: <code>action_id</code> · <code>kind</code> {reverse, mitigate, escalate} · <code>technique?</code> (mitigate: failover reroute throttle scale isolate) · <code>target</code> · <code>team?</code> (escalate: the team paged) · <code>expected_effect</code> · <code>blast_radius</code> · <code>rollback</code> · <code>temporary?</code> + <code>revert_when?</code> · <code>idempotency_key</code> · <code>gated</code> · <code>approval</code> {decision, actor, at} · <code>result</code> · <code>status</code> .
followups[] · status	Durable fixes owned elsewhere {detail, basis, owner} · applied \\ partial \\ failed.

VerifyResult — recovery, confirmed

```
{
  "recovered":      true,                                // confirmed from the USER's side
  "before_after":   "p99 4.2s→260ms, errors 18%→0.2%",  // the proof
  "watch_window":   "15m", "recovery_confidence": 0.95,
  "resolution":     "failover to standby; durable fix = disk replacement", // what the next inci
  "temporary_actions_status": [ { "action_id":"a1", "status":"scheduled" } ], // reverted | sch
  "residual":       [ { "item":"replace disk", "owner":"storage" } ],
  "closed_by":      "j.rivera", "status": "closed"      // closed | held | reopened
}
```

Field	What it means
recovered · before_after NEW	Confirmed from the user's side · the symptom X → Y (the proof).
watch_window? · recovery_confidence? NEW	How long watched / how sure. Optional.

resolution NEW	What <i>actually</i> fixed it — the single field the next incident's suggestions read (the learning loop).
temporary_actions_status[] · residual[]	Each temporary action reverted \\ scheduled (must be clear before close) · open obligations.
closed_by · status	The human who closed it · closed \\ held \\ reopened.

5 · Feedback Mongo · data plane (separate from the run)

The operator's verdict — kept apart from the run record

Definition. A human's judgement about an investigation or run — an outcome, a declared failure, or a correction. Universal across domains.

Why separate from the run: the run record says what the *engine did*; feedback says what the *human thought of it* — mixing them would taint the audit. Its own small document, added after close, queried on its own. It feeds two loops: **outcome** confirms the **resolution** the next incident reads; **failure / correction** lower autonomy on the capability involved (the autonomy ladder).

```
{
  "_id":          "fb-9912",
  "subject":      { "domain":"app-incident", "id":"INC-4821", "kind":"incident" }, // SubjectRef
  "run_id":       "INC-4821:run:1",    // optional
  "actor":        "j.rivera",
  "kind":         "outcome",           // outcome (did it work) | failure (engine wrong) | correction
  "verdict":      "fix held; good call on the failover",
  "note":         "disk replaced next morning, no recurrence",
  "at":           "2026-06-19T09:00Z"
}
```

Field	What it means
subject · run_id?	The SubjectRef {domain, id} the verdict is about — domain-neutral (incident-triage fills it with the incident; another domain with its own subject) · the run (optional).
actor · at	The human giving the verdict · when.
kind	outcome · failure · correction .
verdict · note	The judgement + free context.

6 · Config Postgres · config plane

Shared, versioned, governed: the Playbook and the Capability registry

Config is not per-incident — it is shared across every investigation, versioned, and changed under governance. So it lives **normalized** in Postgres, edited on its own lifecycle. An incident only *references* it (by `playbook@version` and capability id). All of this is universal — the registry serves any playbook.

6.1 · Playbook — the stored, versioned process

Definition. The row form of §1 — the process as markdown plus the machine-readable front-matter, immutably versioned. One row per playbook version; a library of them serves many domains.

Why md + version + fields, normalized (not denormalized): the playbook is the *human* contract — so its body stays **markdown** (`body_md` , exactly the §1 text), readable and diffable. A few structured columns (`version` , `domain` , `status` , `owner`) make it queryable; the machine parts (`phases` , `schemas` , `defaults`) ride along as JSONB. PK is (**id, version**) — every edit is a new immutable row, so a run pins `playbook@version` and old incidents stay reproducible. It is **not** copied into the incident doc — the incident stores only the pin.

```
{
  "id":      "incident-triage",
  "version": "1.0.0",           // PK = (id, version) – every change is a new immutable
  "domain":  "app-incident",    // lets ONE engine carry a library of playbooks (inciden
  "status":  "active",          // draft | active | deprecated
  "owner":   "sre-platform",
  "body_md": "## assess ... ## root-cause ... ## remediation ... ## verify-close ...", // the §1 ma
  "phases":  [ /* front-matter phases: id, effect, needs, output, ... */ ], // jsonb
  "schemas": { /* output_schemas: AssessResult, RootCauseResult, ... */ }, // jsonb
  "defaults": { "on_failure":"run-remaining", "retry":{"max":3,"backoff":"exponential"} }, // j
  "unknown_access": "ask", "changelog": "1.0.0 – initial graph-based playbook"
}
```

Field	What it means
id · version	The composite key. <code>version</code> is semver; a run pins <code>id@version</code> so it is reproducible.
domain	Which problem domain it serves — lets one engine carry many playbooks (the heart of §2).
status · owner	draft · active · deprecated (only one <code>active</code> version dispatched) · the owning team.
body_md	The markdown body (the §1 prose) — the human-readable process the agent follows.
phases · schemas · defaults	JSONB — the front-matter the engine executes (phases + <code>needs</code> / <code>effect</code> , the output contracts, the defaults).
unknown_access · changelog	What to do for an un-policied capability (<code>ask</code>) · why this version exists.

6.2 · Capability registry — how intents become governed calls

Definition. Three small tables that turn a playbook `need` (an intent) into a real, allowed tool call: **Provider** (where capabilities come from), **DeclaredCapability** (what each can do), **CapabilityPolicy** (whether it's

allowed).

Why this is config, not code: the playbook names *intents* (`needs:[metrics, logs, ...]`), never tools. The registry resolves an intent → matching `DeclaredCapability` (by intent + `effect`), and `CapabilityPolicy` decides **allow / ask / deny**. A **human owns the policy** — so what the agent may do is governed data a reviewer edits, not a code change. Shared by every playbook.

Provider — a source of capabilities

```
{
  "id":      "appd",
  "name":    "AppDynamics",
  "kind":    "mcp_remote", // skill|mcp_local|mcp_remote|a2a_agent|api
  "connection":{ "url":"..." }, // jsonb - varies by kind
  "trusted": true,
  "status":  "connected", // registered|connected|disabled|error
  "last_synced":"2026-06-18T09:00Z"
}
```

DeclaredCapability — one action a provider offers

```
{
  "id":      "appd__get_health", // provider__action
  "provider": "appd",
  "description": "Health of an app / tier / node",
  "input_schema": { /* JSON Schema */ },
  "output_schema":{ /* JSON Schema */ },
  "effect_hint": "read", // read | write | unknown
  "intents":    ["telemetry","metrics"] // which playbook `needs` it satisfies - the resolver
}
```

CapabilityPolicy — the human-owned gate (the heart of governance)

```
{
  "capability_id": "bladelogic__restart_service",
  "effect":       "write", // read | write | unknown
  "access":       "ask",   // allow | ask | deny ← what the agent may do
  "status":       "active", // active | pending_review (a NEW capability lands here first
  "reviewed_by":  "sre-lead", "reviewed_at": "2026-06-10"
}
```

Table	Fields
Provider	<code>id</code> · <code>name</code> · <code>kind</code> (<code>skill</code> \ <code>mcp_local</code> \ <code>mcp_remote</code> \ <code>a2a_agent</code> \ <code>api</code>) · <code>connection</code> (<code>jsonb</code>) · <code>trusted</code> · <code>status</code> · <code>last_synced</code> . Where capabilities are discovered + synced from.
DeclaredCapability	<code>id</code> (<code>provider__action</code>) · <code>provider</code> · <code>description</code> · <code>input_schema</code> · <code>output_schema</code> · <code>effect_hint</code> (<code>read</code> \ <code>write</code> \ <code>unknown</code>) · <code>intents[]</code> (which playbook <code>needs</code> it satisfies). The synced contract of one action.
CapabilityPolicy	<code>capability_id</code> · <code>effect</code> · <code>access</code> (<code>allow</code> \ <code>ask</code> \ <code>deny</code>) · <code>status</code> (<code>active</code> \ <code>pending_review</code>) · <code>reviewed_by</code> · <code>reviewed_at</code> . The sticky, human-owned

decision; a new capability starts `pending_review` + `deny` until a human rules.

The link back to §1: phase `needs: [metrics]` → registry finds DeclaredCapabilities tagged *metrics* with a matching `effect` → `CapabilityPolicy` gates each → the chosen call is logged as a `Step{kind: tool_call, capability}` (§4.2). The playbook never names `appd` or `bladelogic` — only the registry does.

7 · Storage — the two planes

Where everything lives — data in Mongo, config in Postgres, run state in LangGraph

Now that the entities are defined, here is where each lives. Three concerns, three homes — kept apart so each is shaped for how it's read.

Plane	Store	Holds	Shape
<code>data</code>	MongoDB (primary)	the subject document — the graph (§3), the phase records + outputs (§4), and feedback (§5)	Denormalized , one doc per subject — the whole subject reads back in a single query (§8). One collection per domain (here <code>incidents</code>).
<code>config</code>	PostgreSQL	the playbook (§6.1) and the capability registry (§6.2)	Normalized , versioned, orderly — config is shared across all incidents and governed independently.
<code>run</code>	LangGraph checkpointer (on Postgres)	the live checkpoint + the conversation — the in-flight run state	The engine's mechanism , not a schema we model. Materializes into the Mongo doc on close.

What we purge, and what we trust. We do **not** hand-model a bespoke run-state schema (no `Session/Run/PhaseRun/Action/RunEvent/Checkpoint` tables) — **LangGraph's checkpointer owns the live run state**, and the durable journey is already captured by the Phase Record (§4), written into the subject document on close. We keep exactly what we trust: the **subject document** (data) and the **playbook + registry** (config). The planes link by the subject — `playbook@version` on the subject doc, and the `{domain, id}` `SubjectRef` from the run.

The live session is runtime, not a schema. The **conversation** (the live chat) and the in-flight run are a runtime layer, not durable shapes — designed in the **Design doc · B8: *free chat* → *shared session***; a **per-session lock** serializes the one run across stateless servers; a **per-session channel** syncs the chat across users. The data model holds only the durable record (graph + phase records + outputs); the chat history is persisted with the run and archived to the subject document on close.

Why split this way: the subject is read *hot* and *whole* during the investigation → denormalized Mongo (one read). Config is *shared, versioned, governed* → normalized Postgres (one source of truth). Run state is *transient and recovery-critical* → LangGraph already solves it; re-modelling it would be duplication to keep in sync.

8 · The Incident document — MongoDB Mongo · data plane

One denormalized document per subject — the whole thing, together

(here, an incident)

Everything from §3–§5 in one place: the graph (nodes + facts + edges), the phase outputs, denormalized so the subject reads back in a single query. It references config by `playbook@version`; the live run state stayed in LangGraph (§7). (Example: *failed disk* → *RAID rebuild* → *DB I/O* → *app*.)

```
// the SUBJECT document – one collection per domain; here db.incidents (domain app-incident).
// _id is unique within the domain; the GLOBAL subject identity = (domain, _id) – the SubjectRef
{
  "_id": "INC-4821", "domain": "app-incident", // → global subject = (domain, _id); nodes con
  "incident_type": "performance",
  "symptom": "checkout p99 0.4s→4.2s, errors 0.5%→18%",
  "priority": "P1", "state": "closed", "owner_team": "payments-sre",
  "playbook": "incident-triage@1.0.0", // the only link to the config plane (§6.1)
  "opened_at": "2026-06-18T14:02Z", "closed_at": "2026-06-18T14:40Z",
  "impact_assessment": { "scope": "payments-api / checkout – customer-facing",
    "blast_radius": ["app:payments-api", "biz:checkout-journey"], "severity": "P1", "urgency": "high"
    "business_impact": "checkout payments degraded; revenue path" },

  "graph": {
    "nodes": [
      { "id": "app:payments-api", "kind": "system", "type": "app", "layer": "app", "labels": ["payments",
        "facts": [ { "key": "health", "value": "degraded", "source": "appd", "evidence_ref": "appd://paym
      { "id": "db:payments-ora", "kind": "system", "type": "database", "layer": "database",
        "facts": [ { "key": "io_wait_ms", "value": 28, "source": "oem", "evidence_ref": "oem://payments-o
      { "id": "stor:pay-vol", "kind": "system", "type": "storage", "layer": "storage", "labels": ["volume
        "facts": [ { "key": "latency_ms", "value": 22.4, "impact_state": "degraded", "source": "netapp", "
      { "id": "INC-4821", "kind": "incident", "type": "incident" }, // the ONE subje
      { "id": "chg:deploy-rev47", "kind": "change", "type": "change", "labels": ["deploy"], "props": { "at
      { "id": "alert:7741", "kind": "alert", "type": "alert", "props": { "status": "resolved", "fingerprin
    ],
    "edges": [
      { "type": "depends_on", "from": "app:payments-api", "to": "db:payments-ora", "props": { "via": "tra
      { "type": "hosted_on", "from": "db:payments-ora", "to": "stor:pay-vol" },
      { "type": "suspected_cause", "from": "INC-4821", "to": "stor:pay-vol",
        "props": { "confidence": { "value": 0.9, "basis": "evidence-complete + matches INC-4820"}, "ran
        "path": ["app:payments-api", "db:payments-ora", "stor:pay-vol"] } },
      { "type": "related_to", "from": "INC-4821", "to": "INC-4820", "props": { "kind": "similar" } } //
    ]
  },

  "phases": [
    { "phase": "root-cause", "output": { "candidates": [ { "cause": "pay-vol disk failure → RAID rebuild
```

```

    "confidence":{"value":0.9,"basis":"evidence-complete + matches INC-4820"},"rank":1,
    "path":["app:payments-api","db:payments-ora","stor:pay-vol"]}], "selected":0, "status":
{ "phase":"remediation","output":{"actions":[
  {"action_id":"a1","kind":"mitigate","technique":"failover","target":"db:payments-ora",
    "expected_effect":"DB served by standby; I/O normal","blast_radius":"payments read path",
    "rollback":"fail back once disk replaced","temporary":true,"revert_when":"incident_clos",
    "approval":{"decision":"approve","actor":"j.rivera","at":"14:21"},"result":"applied; I/
  {"action_id":"a2","kind":"escalate","team":"storage","result":"DISK ticket opened","stat",
    "followups":[{"detail":"replace disk 1.4.7","basis":"escalate","owner":"storage"}]}, "stat
{ "phase":"verify-close","output":{"recovered":true,"before_after":"p99 4.2s-260ms, errors",
  "resolution":"failover to standby; durable fix = disk replacement","temporary_actions_st",
  "closed_by":"j.rivera","status":"closed" } }
}
}

```

Indexes: `_id · incident_type · graph.nodes.id · graph.edges.type · related_to` + a fingerprint/embedding index for `similar` -search. • **Two links, no duplication:** `playbook@version` → config (§6); the `{domain, id}` SubjectRef ← the LangGraph run, which writes its final graph + phase outputs through to this document on close.

Appendix · Coverage vs HLD / LLD

Every HLD/LLD entity accounted for — nothing dropped silently

Cross-check against the design docs. **kept** = present as-is; **superseded** = consciously folded into a simpler trusted form (5 of them), with the reason.

HLD / LLD entity	Status	Where it lives now / why
Provider · DeclaredCapability · CapabilityPolicy	kept	§6.2 — as-is.
Playbook (+ Phase)	kept	§1 (the file) + §6.1 (the stored row).
Incident (+ parent_incident, links, topology, affected_services)	kept	§8 doc + §3 graph; <code>parent_incident / links</code> → <code>related_to {child related caused_by chained similar}</code> ; <code>topology</code> → the graph.
AssessResult · RootCauseResult · RemediationResult · VerifyResult	kept	§4.4 — this playbook's output contracts.
Action	kept	§4.2 — a <code>Step{kind:tool_call}</code> .
HumanInteraction	kept	§4.2 — a <code>Step{kind:decision}</code> + <code>RemediationResult.approval</code> .
Feedback	kept	§5 — separate from the run.
Evidence (entity)	superseded	→ <code>Fact.evidence_ref</code> + <code>Step.evidence[]</code> (deep links). The backing data stays in the source tool — a separate copy

		would just drift.
RunEvent	superseded	→ the <code>Step</code> log is the event stream (one ordered journey, no second table).
TimelineEvent	superseded	→ a UI <i>projection</i> of <code>PhaseRecord.steps</code> — derived, not stored.
Session · Run · PhaseRun(exec) · Checkpoint · Lease	superseded	→ the run plane = LangGraph checkpointer (§7); the durable journey = §4. No bespoke run schema.
SLO · burn_rate fields	superseded	removed by design — kept the model generic (impact stated in business terms, not SLO-bound).
Enums (Severity, *Status, Effect, Access, ...)	kept	inline in the relevant schemas above.

Verdict: coverage complete. Eight entity groups kept as-is; five consciously superseded to keep the model KISS and trusted — each named here, none dropped silently.

Reads as the system flows: **playbook** (agreed anchor) → **domain-neutral** (one engine, many playbooks) → **graph + phase record + feedback** (the per-incident data) → **config** (playbook + registry) → **storage** (where each lives) → **incident doc** (the consolidated read-model) → **coverage**. Each schema is defined, explained, shown as JSON, and tabled. · Full playbook: `incident-triage-playbook-v1.md` · validation trail: `working-model.md` · consolidated + coverage-checked from the HLD + LLD.